23

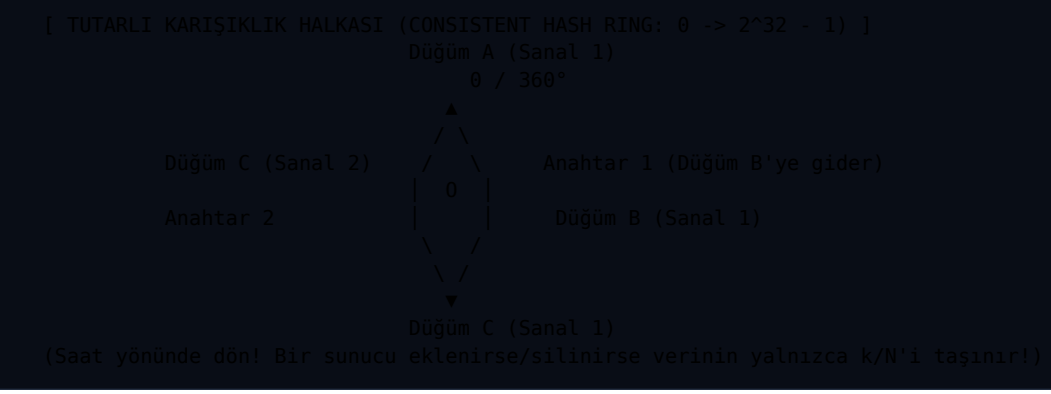
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Tutarlı Karışıklık (Consistent Hashing) & Sanal Düğümler (Virtual Nodes)

Consistent Hashing & Virtual Nodes: The Elastic Scaling Ring

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Geleneksel modülo aritmetiğinde (`Hash % N`), 4 sunucunuz varken 5. sunucuyu eklerseniz sınıftaki herkesin sırası değişir; tüm önbellek geçersiz kalır ve veritabanı çöker! Consistent Hashing ise yuvarlak bir yemek masası gibidir (0'dan 360 dereceye bir çember). Sunucuları masanın etrafına rastgele oturtursunuz. Yeni gelen bir yemeği masanın üzerine koyup saat yönünde döndürürsünüz; yemeği ilk gören sunucu onu afiyetle yer! Masaya yeni bir sandalye koysanız bile sadece o sandalyenin hemen yanındaki birkaç yemek yer değiştirir; geri kalan herkes yemeğini yemeye devam eder!

🇬🇧 ENGLISH (ELI5)

Under naive modular hashing (`hash % N`), expanding from 4 to 5 servers scrambles the destination of 80% of all keys, triggering a total cache wipeout! Consistent Hashing is an infinite circular conference table (0 to 2^32 - 1). You position servers at discrete degrees around the circle. To store a key, hash it to a point on the circle and walk clockwise: the first server node you bump into claims the key! If a new server pulls up a chair, only its immediate neighbor hands over a sliver of keys; the remaining 80% of data stays completely untouched!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Consistent Hashing (Karger ve arkadaşları, MIT 1997), dağıtık önbellek (Memcached, Akamai CDN) ve NoSQL (Amazon DynamoDB, Apache Cassandra) mimarilerinin kalbidir. Avantajı: Kümedeki düğüm sayısı N değiştiğinde, taşınması gereken anahtar sayısı geleneksel mimarideki gibi %100 değil, sadece ortalama k/N kadardır (k = toplam anahtar). Sanal Düğümler (Virtual Nodes / Vnodes): Fiziksel bir sunucunun halka üzerinde tek bir noktada durması 'Hotspot' ve dengesiz yük dağılımına yol açar. Çözüm: Her fiziksel sunucuya halka üzerinde 100-256 adet sanal etiket atanır (`NodeA-1`, `NodeA-2`...). Bu sayede yük çember boyunca mükemmel bir homojenlikle dağılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Consistent Hashing maps both servers and data keys onto a continuous hash ring (typically 0 to 2^32 - 1). Keys route clockwise to the first encountered server token. When node scaling occurs, only k/N keys migrate (k = total keys, N = server count), avoiding mass cache evictions. The Virtual Nodes (Vnodes) Innovation: A physical server represented by a single token risks severe load skew due to random clustering. By projecting each physical node into 100-256 virtual points across the ring perimeter, keys distribute uniformly across hardware proportional to host capacity.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Sanal Düğümlü (Vnodes) Consistent Hash Halkası Implementasyonu
import java.util.*;

public class ConsistentHashRing<T> {
    private final int numberOfReplicas; // Sanal düğüm sayısı
    private final SortedMap<Integer, T> circle = new TreeMap<>();

    public ConsistentHashRing(int numberOfReplicas, Collection<T> nodes) {
        this.numberOfReplicas = numberOfReplicas;
        for (T node : nodes) addNode(node);
    }

    public void addNode(T node) {
        for (int i = 0; i < numberOfReplicas; i++) {
            circle.put(hash(node.toString() + "-vnode-" + i), node);
        }
    }

    public T getNode(String key) {
        if (circle.isEmpty()) return null;
        int hash = hash(key);
        if (!circle.containsKey(hash)) {
            SortedMap<Integer, T> tailMap = circle.tailMap(hash);
            hash = tailMap.isEmpty() ? circle.firstKey() : tailMap.firstKey(); // Saat yönünde dön
        }
        return circle.get(hash);
    }

    private int hash(String val) {
        return val.hashCode() & 0x7fffffff;
    }
}

void main() {
    var ring = new ConsistentHashRing<String>(3, List.of("Server-A", "Server-B", "Server-C"));
    IO.println("Key 'user_9912' Hangi Sunucuda? -> " + ring.getNode("user_9912"));
    IO.println("Key 'order_441' Hangi Sunucuda?  -> " + ring.getNode("order_441"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta sanal düğümleri (Virtual Nodes) anlatmayı unutursanız mülakatçı 'Düğümler halkaya rastgele düşerse biri aşırı yüklenip çökmez mi?' diye soracaktır. Sanal düğümlerin varyansı düşürüp yükü homojenleştirdiğini doğrudan siz söyleyin.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Failing to proactively propose Virtual Nodes (Vnodes) invites the interviewer to challenge you on hash skew. Preemptively demonstrate that virtual tokens equalize ring dispersion and reflect asymmetric server hardware.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Consistent Hashing, sunucu ekleme/çıkarmada veri taşıma maliyetini k/N seviyesine indirir.
- Sanal Düğümler (Virtual Nodes) çember üzerindeki yükü mükemmel dengeler ve hotspot'u önler.
- DynamoDB, Cassandra ve Discord gibi devasa dağıtık sistemlerin temel omurgasıdır.

🎯 Key Takeaways for the Interview

- Consistent Hashing bounds key migration to k/N upon node churn, preserving cache stability.
- Virtual Nodes neutralize non-uniform distribution skew across heterogeneous machines.
- Serves as the structural backbone of Amazon DynamoDB, Apache Cassandra, and Discord.